

Adding LLMOps to the LLM Twin

In the previous section, we saw how to set up the infrastructure for the LLM Twin project by manually building the Docker image and pushing it to ECR. We want to automate the entire process and implement a CI/CD pipeline using GitHub Actions and a CT pipeline using ZenML. As mentioned earlier, implementing a CI/CD/CT pipeline ensures that each feature pushed to main branches is consistent and tested. Also, by automating the deployment and training, you support collaboration, save time, and reduce human errors.

Finally, at the end of the section, we will show you how to implement a prompt monitoring pipeline using Opik from Comet ML and an alerting system using ZenML. This prompt monitoring pipeline will help us debug and analyze the RAG and LLM logic. As LLM systems are non-deterministic, capturing and storing the prompt traces is essential for monitoring your ML logic.

Before diving into the implementation, let's start with a quick section on the LLM Twin's CI/CD pipeline flow.

LLM Twin's CI/CD pipeline flow

We have two environments: staging and production. When developing a new feature, we create a new branch out of the staging branch and develop solely on that one. When we are done and consider the feature finished, we open a **pull request (PR)** to the staging branch. After the feature branch is accepted, it is merged into the staging branch. This is a standard workflow in most software applications. There might be variations, like adding a dev environment, but the principles remain the same.

As illustrated in *Figure 11.14*, the CI pipeline is triggered when the PR opens. At this point, we test the feature branch for linting and formatting errors. Also, we run a `gitleaks` command to check for credentials and sensitive information that was committed by mistake. If the linting, formatting, and `gitleaks` steps pass (also known as static analysis), we run the automated tests. Note that the static analysis steps run faster than the automated tests. Thus, the order matters. That's why adding the static analysis steps at the beginning of the CI pipeline is good practice. We propose the following order of the CI steps:

- `gitleaks` checks
- Linting checks
- Formatting checks
- Automated testing, such as unit and integration tests

If any check fails, the CI pipeline fails, and the developer who created the PR cannot merge it into the staging branch until it fixes the issues.

Implementing a CI pipeline ensures that new features follow the repository's standards and don't break existing functionality. The exact process repeats when we plan to merge the staging branch into the production one. We open a PR, and the CI pipeline is automatically executed before merging the staging branch into production.

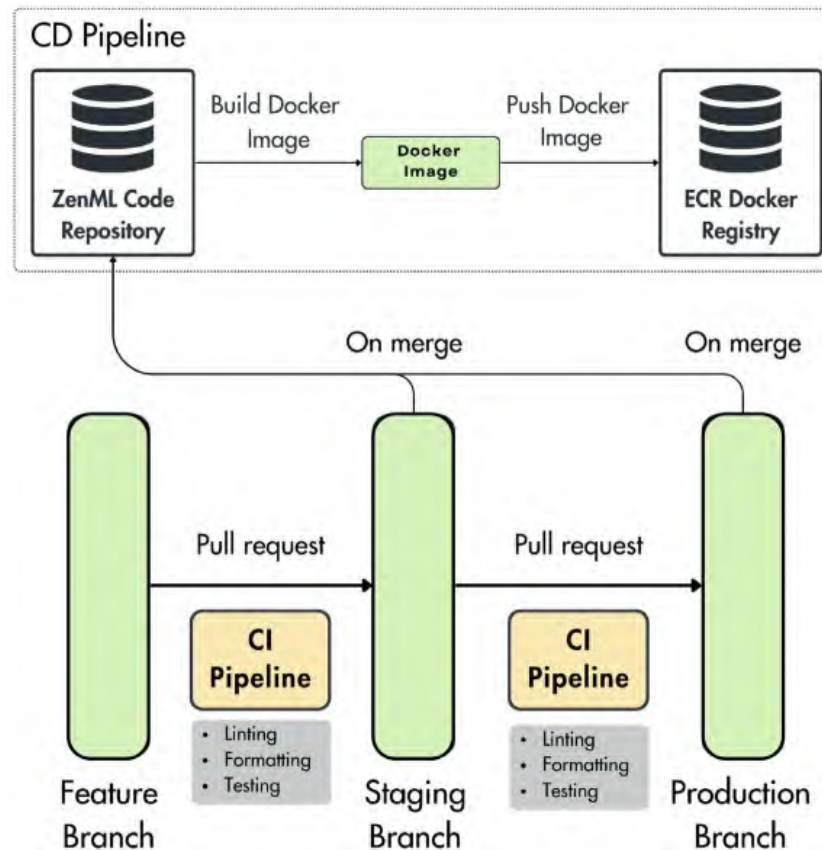


Figure 11.14: CI/CD pipelines flow

The CD pipeline runs after the branch is merged. For example, after the feature branch is merged into staging, the CD pipeline takes the code from the staging branch, builds a new Docker image, and pushes it to the AWS ECR Docker repository. When running future pipeline runs in the staging environment, it will use the latest Docker image that was built by the CD pipeline. The exact process happens between staging and production. Still, the key difference is that the staging environment exists as an experimental place where the QA team and stakeholders can further manually test the new feature along with what is automatically tested in the CI pipeline.



In our repository, we used only a main branch, which reflects production, and feature branches to push new work. We did this to keep things simple, but the same principles apply. To extend the flow, you must create a staging branch and add it to the CD pipeline.

More on formatting errors

Formatting errors relate to the style and structure of your code, ensuring that it adheres to a consistent visual layout. This can include the placement of spaces, indentation, line length, and other stylistic elements.

The main purpose of formatting is to make your code more readable and maintainable. Consistent formatting helps teams work together more effectively, as the code looks uniform, regardless of who wrote it. Examples of formatting errors are:

- Incorrect indentation (e.g., mixing spaces and tabs)
- Lines that are too long (e.g., exceeding 79 or 88 characters, depending on your style guide)
- Missing or extra spaces around operators or after commas

More on linting errors

Linting errors relate to potential issues in your code that could lead to bugs, inefficiencies, or non-adherence to coding standards beyond just style. Linting checks often involve static analysis of the code to catch things like unused variables, undefined names, or questionable practices.

Linting's main goal is to catch potential errors or bad practices early in the development process, improving code quality and reducing the likelihood of bugs. Examples of linting errors are:

- Unused imports or variables
- Undefined variables or functions are being used
- Potentially dangerous code (e.g., using `==` instead of `is` for checking against `None`)

We use Ruff, a versatile tool for formatting and linting. It incorporates checks for common formatting issues and PEP 8 compliance, as well as deeper linting checks for potential errors and code quality problems. Also, it is written in Rust, making it fast for big codebases.

Before implementing what we've explained above, let's examine the core principles of GitHub Actions.

Quick overview of GitHub Actions

GitHub Actions is a CI/CD platform provided by GitHub that allows developers to automate their workflows directly within a GitHub repository. It enables users to build, test, and deploy their code directly from GitHub by defining workflows in YAML files. Since it's part of GitHub, it works seamlessly with repositories, issues, PRs, and other GitHub features. Here are the key components you should know about:

- **Workflows:** A workflow is an automated process defined in a YAML file located in your repository's `.github/workflows` directory. It specifies what should happen (e.g., build, test, and deploy) and when (e.g., on push, on PR).
- **Jobs:** Workflows are made up of jobs, which are groups of steps that execute on the same runner. Each job runs in its own virtual environment.
- **Steps:** Jobs are made up of multiple independent steps, which can be actions or shell commands.
- **Actions:** Actions are reusable commands or scripts. You can use pre-built actions from GitHub Marketplace or create your own. You can think of them as Python functions.
- **Runners:** Runners are the servers that run your jobs. GitHub provides hosted runners (Linux, Windows, macOS), or you can even self-host your runners.

A workflow is described using YAML syntax. For example, a simple workflow that clones the current GitHub repository and installs Python 3.11 on an Ubuntu machine looks like this:

```
name: Example
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v3

      - name: Setup Python
        uses: actions/setup-python@v3
        with:
          python-version: "3.11"
```

The workflows are triggered by events like `push`, `pull_request`, or `schedule`. For example, you might trigger a workflow every time code is pushed to a specific branch. Now that we understand how GitHub Actions works, let's look at the LLM Twin's CI pipeline.

The CI pipeline

The LLM Twin's CI pipeline is split into two jobs:

- A **QA job** that looks for formatting and linting errors using `Ruff`. Also, it runs a `gitleaks` step to scan for leaked secrets throughout our repository.
- A **test job** that runs all our automatic tests using `Pytest`. In our use case, we implemented just a dummy test to showcase the CI pipeline, but using the structure from this book, you can easily extend it with real tests for your use case.

GitHub Actions CI YAML file

The YAML file sits under `.github/workflows/ci.yaml`. It begins by defining the workflow's name as `CI`, as you can see in the following snippet. This label will be used to identify the workflow within GitHub's Actions interface. Next, the section specifies that the workflow should be triggered whenever a `pull_request` event occurs. Hence, the CI workflow will automatically run whenever a PR is opened, synchronized, or reopened.

```
name: CI

on:
  pull_request:
```

The `concurrency` section ensures that only one instance of this workflow runs for a given reference (like a branch) at any given time. The `group` field is defined using GitHub's expression syntax to create a unique group name based on the workflow and the reference. The `cancel-in-progress: true` line ensures that if a new workflow run is triggered before the previous one finishes, the previous run is canceled. This is particularly useful to prevent redundant executions of the same workflow.

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

The workflow defines two separate jobs: `qa` and `test`. Each job runs on the latest version of Ubuntu, specified by `runs-on: ubuntu-latest`.

The first job, named QA, is responsible for quality assurance tasks like code checks and formatting verification. Within the qa job, the first step is to check out the repository's code using the `actions/checkout@v3` action. This step is necessary to ensure that the job has access to the code that needs to be analyzed.

```
jobs:
  qa:
    name: QA
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v3
```

The next step is to set up the Python environment. This is done using the `actions/setup-python@v3` action, with the Python version specified as "3.11". This step ensures that the subsequent steps in the job will run in the correct Python environment.

```
- name: Setup Python
  uses: actions/setup-python@v3
  with:
    python-version: "3.11"
```

The workflow then installs Poetry using the `abatilo/actions-poetry@v2` action, specifying the version of Poetry as 1.8.3:

```
- name: Install poetry
  uses: abatilo/actions-poetry@v2
  with:
    poetry-version: 1.8.3
```

Once Poetry is set up, the workflow installs the project's development dependencies using the `poetry install --only dev` command. Additionally, the workflow adds the `poethepoet` plugin for Poetry, which will be used to run predefined tasks more conveniently within the project.

```
- name: Install packages
  run: |
    poetry install --only dev
    poetry self add 'poethepoet[poetry_plugin]'
```

The qa job then runs several quality checks on the code. The first check uses a tool called `gitleaks` to scan for secrets in the codebase, ensuring that no sensitive information is accidentally committed:

```
- name: gitleaks check
  run: poetry poe gitleaks-check
```

Following the `gitleaks` check, the workflow runs a linting process to enforce coding standards and best practices in the Python code. This is achieved through the `poetry poe lint-check` command, which uses `Ruff` under the hood.

```
- name: Lint check [Python]
  run: poetry poe lint-check
```

The last step in the qa job is a format check, which ensures that the Python code is properly formatted according to the project's style guidelines. This is done using the `poetry poe format-check` command, which uses `Ruff` under the hood.

```
- name: Format check [Python]
  run: poetry poe format-check
```

The **second job** defined in the workflow is the test job, which also runs on the latest version of Ubuntu. Like the qa job, it starts by checking out the code from the repository and installing Python 3.11 and Poetry 1.8.3.

```
test:
  name: Test
  runs-on: ubuntu-latest

  steps:
    - name: Checkout
      uses: actions/checkout@v3
    ...
```

After setting up the system dependencies, the test job installs all the project's dependencies with the `poetry install` command. As we want to run the tests, this time, we need to install all the dependencies that are required to run the application.

```
- name: Install packages
  run: |
```

```
poetry install --without aws
poetry self add 'poethepoet[poetry_plugin]'
```

Finally, the test job runs the project's tests using the `poetry poe test` command. This step ensures that all tests are executed and provides feedback on whether the current code changes break any functionality.

```
- name: Run tests
  run: |
    echo "Running tests..."
    poetry poe test
```

If any of the steps from the QA or test jobs fail, the GitHub Actions workflow will fail, resulting in the PR not being able to be merged until the issue is fixed. By taking this approach, we ensure that all the new features added to the main branches respect the standard of the project and that it doesn't break existing functionality through automated tests.

Figure 11.15 shows the CI pipeline in the **Actions** tab of the GitHub repository. It was run after a commit with the message **feat: Add Docker image and CD pipeline** and ran the two jobs described above, QA and Test.

The screenshot shows the GitHub Actions interface for the repository 'PacktPublishing / LLM-Engineering'. The 'Actions' tab is selected, displaying a workflow run titled 'feat: Add Docker image and CD pipeline #49' with a green checkmark indicating success. The workflow was triggered via a pull request 3 days ago. The status is 'Success' and the total duration is '2m 15s'. The workflow file is 'ci.yaml' with the trigger 'on: pull_request'. The jobs listed are 'QA' (18s) and 'Test' (2m 7s), both marked as successful with green checkmarks.

Triggered via	Status	Total duration
pull request 3 days ago	Success	2m 15s

Job	Duration
QA	18s
Test	2m 7s

Figure 11.15: GitHub Actions CI pipeline run example

The CD pipeline

The CD pipeline will automate the Docker steps we manually performed in the **Deploying the LLM Twin’s pipelines to the cloud** section, which are:

- Set up Docker.
- Log in to AWS.
- Build the Docker image.
- Push the Docker image to AWS ECR.

With that in mind, let’s look at the GitHub Actions YAML file, which sits under `.github/workflows/cd.yaml`. It begins by naming the workflow CD and specifying the trigger for this workflow. The trigger is any push to the repository’s main branch. This workflow will automatically run when new code is pushed to the main branch, usually when a PR is merged into the main branch. The `on.push` configuration sets up the trigger:

```
name: CD

on:
  push:
    branches:
      - main
```

The workflow then defines a single job named `Build & Push Docker Image`:

```
jobs:
  build:
    name: Build & Push Docker Image
    runs-on: ubuntu-latest
```

The first step within the job is to check out the repository’s code.

```
steps:
  - name: Checkout Code
    uses: actions/checkout@v3
```

After checking out the code, the workflow sets up `docker buildx`, a Docker CLI plugin that extends Docker’s build capabilities with features like multi-platform builds and cache import/export:

```
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3
```

The next step involves configuring the AWS credentials. This step is crucial for interacting with AWS services, such as Amazon **Elastic Container Registry (ECR)**, where the Docker images will be pushed. The AWS access key, secret access key, and region are securely retrieved from the repository's secrets to authenticate the workflow with AWS. This ensures the workflow has the necessary permissions to push Docker images to the ECR repository. We will show you how to configure these secrets after wrapping up with the YAML file:

```
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v1
  with:
    aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
    aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
    aws-region: ${ secrets.AWS_REGION }
```

Once the AWS credentials are configured, the workflow logs in to Amazon ECR. This step is essential for authenticating the Docker CLI with the ECR registry, allowing subsequent steps to push images to the registry:

```
- name: Login to Amazon ECR
  id: login-ecr
  uses: aws-actions/amazon-ecr-login@v1
```

The final step in the workflow involves building the Docker image and pushing it to the Amazon ECR repository. This is accomplished using the `docker/build-push-action@v6` action. The `context` specifies the build context, which is typically the repository's root directory. The `file` option points to the Dockerfile, which defines how the image should be built. The `tags` section assigns tags to the image, including the specific commit SHA and the latest tag, which is a common practice for identifying the most recent version of the image. The `push` option is set to `true`, meaning the image will be uploaded to ECR after it is built:

```
- name: Build images & push to ECR
  id: build-image
  uses: docker/build-push-action@v6
  with:
    context: .
    file: ./Dockerfile
    tags: |
      ${ steps.login-ecr.outputs.registry }/${ secrets.AWS_ECR_NAME }
      ${ github.sha }
```

```

    ${{ steps.login-ecr.outputs.registry }}/${{ secrets.AWS_ECR_NAME
  }}:latest
  push: true

```

To conclude, the CD pipeline authenticates to AWS, builds the Docker image, and pushes it to AWS ECR. The Docker image is pushed with `latest` and the commit's SHA tag. By doing so, we can always use the latest image and point to the commit of the code from which the image was generated.

Also, in our code, we have only a `main` branch, which reflects our production environment. But you, as a developer, have the power to extend this functionality with a staging and dev environment. You just have to add the name of the branches in the `on.push.branches` configuration at the beginning of the YAML file.

In *Figure 11.16*, you can observe how the CD pipeline looks after a PR is merged into the production branch. As seen before, we only have the **Build & Push Docker Image** job here.

The screenshot shows the GitHub Actions interface for the repository `PacktPublishing / LLM-Engineering`. The `Actions` tab is selected, showing a workflow named `CD`. A specific run is highlighted: `Merge pull request #4 from PacktPublishing/zenml-deploy #9`. The run status is `Success`, triggered 21 minutes ago by user `iusztinpaul` on the `main` branch. The total duration is `7m 10s` and there is `1` artifact.

On the left sidebar, under `Jobs`, the job `Build & Push Docker Image` is listed with a green checkmark. Under `Run details`, the `cd.yaml` file is shown with the trigger `on: push`. A summary box at the bottom indicates the job `Build & Push Docker Im...` completed in `7m 0s`.

Figure 11.16: GitHub Actions CD pipeline run example

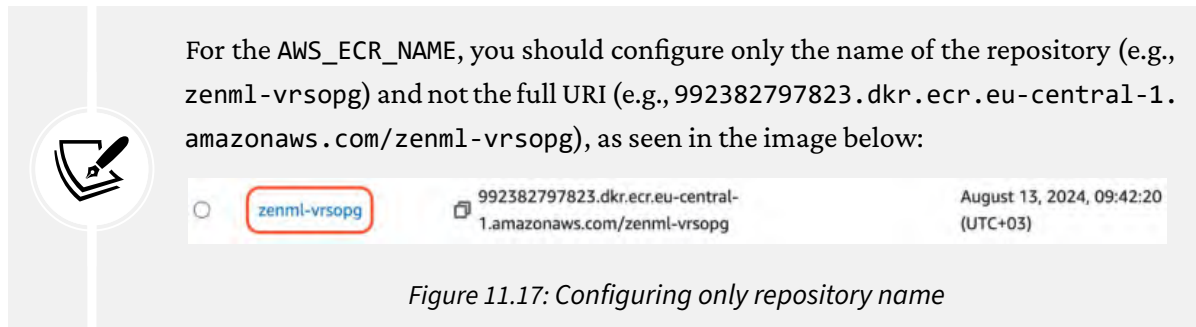
The last step in setting up the CI/CD pipeline is to test it and see how it works.

Test out the CI/CD pipeline

To test the CI/CD pipelines yourself, you must fork the LLM-Engineering repository to have full *write* access to the GitHub repository. Here is the official tutorial on how to fork a GitHub project: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/fork-a-repo>

The last step is to set up a few secrets that will allow the CD pipeline to log in to AWS and point to the right ECR resource. To do so, go to the **Settings** tab at the top of the forked repository in GitHub. In the left panel, in the **Security** section, click on the **Secrets and Variables** toggle and, finally, on **Actions**. Then, on the **Secrets** tab, create four repository secrets, as shown in *Figure 11.17*. These secrets will be securely stored and accessible only by the GitHub Actions CD pipeline.

The `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` are the AWS credentials you used across the book. In *Chapter 2*, you see how to create them. The `AWS_REGION` (e.g., `eu-central-1`) and `AWS_ECR_NAME` are the same ones used in the **Deploying the LLM Twin's pipelines** to the cloud section.



To trigger the CI pipeline, create a feature branch, modify the code or documentation, and create a PR to the main branch. To trigger the CD pipeline, merge the PR into the main branch.